

Polimorfizem

- Vključitveni (inclusion polymorphism)
- Operacijski (operation polymorphism)
- Parametrični (parametric)

Run-time polymorphism vs. **compile-time** polymorphism

1

Vključitveni polimorfizem (run-time)

Neka spremenljivka (referenca) lahko predstavlja objekte različnih razredov:

- iz iste razredne hierarhije
- so primerki razredov, ki implementirajo isti vmesnik

Dejanska metoda, ki se izvede, se določi šele v času izvajanja!

```
Instrument ins;
ins = new Instrument();
ins = new Kitara();
ins = new ElektricnaKitara();

ins.oglasiSe ();
```

```
INalozba xx;
xx = new Nepremicnina ();
xx = new VarcevalniRacun (...);
xx = new KriptoValuta ();

xx.vrniDonosnost ();
xx.vrniTveganje ();
xx.vrniTrenutnoVrednost ();
```

2

Operacijski polimorfizem

- isto ime metode, različni sezname argumentov
- metode se ne morejo razlikovati glede na rezultat, ki ga vračajo

"overloading"

tipično – več konstruktorjev

```
public class OperacijskiPolimorfizem {

    void sestej(int i) {}
    // int sestej(int d) { return d; } ne more biti samo ime drugačno
    // int sestej(int i) {} ne more se razlikovati glede na tip rezultata

    void sestej(int a, int b) {
        System.out.println("ce samo dva int parametra ");
    }

    void sestej(int j, int i, int... dodatni) {
        int skupno = j + i;
        System.out.println(dodatni.length);
        for (int k = 0; k < dodatni.length; k++)
            skupno = skupno + dodatni[k];
        System.out.println("Sestevek " + skupno);
    }

    void sestej(String x) {}
    void sestej(double x) {}
}
```

3

Parametrični polimorfizem

Generiki (generics), parametrizirani razredi

```
public class Sklad<Tip>{
    private Tip[] elementi;
    private int kaz;

    public Sklad(Tip[] elementi) {}

    public void push(Tip element) throws IzjemaPolnSklad {}

    public Tip pop() {}
}

Sklad<String> imena = new Sklad<String>(new String[3]);

Sklad<Delavec> zaposleni = new Sklad<Delavec>(new Delavec[100]);
```

4

Dodatno lahko podamo omejitve glede tipa

```
public class Sklad<Tip extends BancniRacun> {

    Sklad<VarcevalniRacun> vezave = new Sklad<VarcevalniRacun>(new VarcevalniRacun[10]);
    Sklad<TransakcijskiRacun> transakcijski = new Sklad<TransakcijskiRacun>(new TransakcijskiRacun[10]);
    Sklad<BancniRacun> racuni = new Sklad<BancniRacun>(new BancniRacun[100]);

    public class Sklad<Tip extends BancniRacun> {

        private Tip[] elementi;
        private int kaz;

        public Sklad(Tip[] elementi) {}

        public void push(Tip element) throws IzjemaPolnSklad {
            System.out.println("Na skladu dodatno:" + element.vrniStanje());
            // //... elementi dodati
        }

        public class Sklad<Tip extends INalozba> {

            Sklad<VarcevalniRacun> vezave = new Sklad<VarcevalniRacun>(new VarcevalniRacun[10]);
            Sklad<Nepremicnina> nepremicnine = new Sklad<Nepremicnina>(new Nepremicnina[10]);
            Sklad<NavadneDelnice> delnice = new Sklad<NavadneDelnice>(new NavadneDelnice[10]);
```

5

```
public class Sklad<Tip1, Tip2> {

    public class Sklad<Tip extends BancniRacun> {

        public class Sklad<Tip extends NevarnoPosasten> {

            public class Sklad<Tip extends BancniRacun & NevarnoPosasten> {

                // ...

            }

        }

    }

    public class VarcevalniRacun extends BancniRacun implements INalozba, Comparable<VarcevalniRacun> {
```

6

OO koncepti

- objekt ("object")
- ograjevanje ("encapsulation")
- razred ("class")
- vmesnik ("interface")
- dedovanje ("inheritance")
- delegiranje ("delegation")
- komunikacija s sporočili
- polimorfizem ("polymorphism")

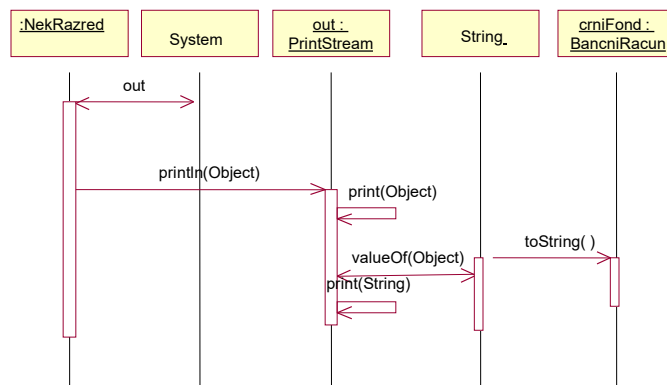
7

Komunikacija s sporočili

```
BancniRacun crniFond; crniFond = new BancniRacun(100000);
```

:

```
System.out.println(crniFond);
```



8

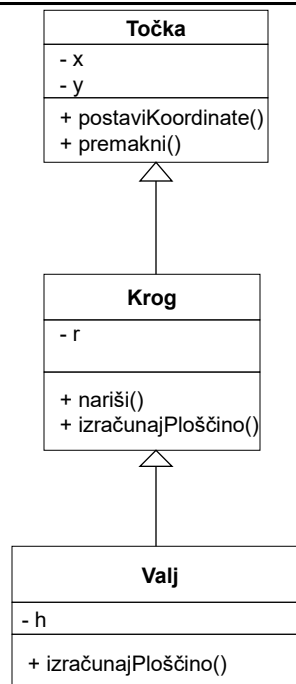
Delegiranje

- Objekt za izvedbo operacije zadolži drug objekt (preloži odgovornost)
- Dogodkovni modeli z delegiranjem
- Delegiranje je pogosto primernejši koncept kot dedovanje (med razredi)
- Kako ugotovimo, če smo dedovanje ustrezno uporabili?
 - Preberemo: **Je** ("is-a")
 - Števnost (**1:**

9

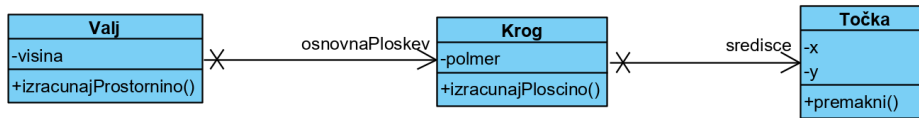
Konceptualna pravilnost

- Preberimo!!!
Krog **je** točka
Valj **je** krog
"is-a"
- Števnost
(kardinalnost) **1:1**



10

Pravilneje z delegiranjem



```

class Krog {
    private Točka sredisce;
    ...
    public double izracunajPloscino() {...}
    ...
}

class Valj {
    private Krog osnovnaPloskev;
    ...
    public double izracunajProstornino() {
        return visina * osnovnaPloskev.izracunajPloscino();
    }
}
  
```

11

OBRAVNAVA IZJEM

12

```

open the file;
if (theFileIsOpen) {
    determine the length of the file;
    if (gotTheFileLength) {
        allocate that much memory;
        if (gotEnoughMemory) {
            read the file into memory;
            if (readFailed) {
                errorCode = -1;
            }
        } else {
            errorCode = -2;
        }
    } else {
        errorCode = -3;
    }
    close the file;
    if (theFileDintClose && errorCode == 0) {
        errorCode = -4;
    } else {
        errorCode = errorCode and -4;
    }
} else {
    errorCode = -5;
}

readFile {
    try {
        open the file;
        determine its size;
        allocate that much memory;
        read the file into memory;
        close the file;
    } catch (fileOpenFailed) {
        doSomething;
    } catch (sizeDeterminationFailed) {
        doSomething;
    } catch (memoryAllocationFailed) {
        doSomething;
    } catch (readFailed) {
        doSomething;
    } catch (fileCloseFailed) {
        doSomething;
    }
}

```

13

```

try {
    PrintWriter pw = new PrintWriter("a.txt");
    k = 10/i;
    System.out.println(hiska.vrniDonosnost());
    args[0] = "Novi";

    i=0;
    k = 10/i;
} catch (NullPointerException ee) {
    System.out.println("Null pointer");
    ee.printStackTrace();
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println(" Tezave z indeksi");
} catch (ArithmeticException e) {
    System.out.println("Ujel sem " + e.getMessage());
} catch (FileNotFoundException e) {
    System.out.println(" teyave pri odpiranju");
} catch (Exception e) {
    // najbolj splosne izjeme na koncu
}

```

14

Tipi izjem

- **Napake (Error)**
za Java VM in HW probleme kot so *InternalError*, *OutOfMemoryError* ipd., teh napak običajno ne prožimo, niti lovimo
- **Izjeme (Exception)**
so “pričakovane” oz. najavljene izjeme
torej morebitne težave, ki jih lahko pričakujemo, t.i. checked exception
- **RT izjeme (RuntimeException)**
tipično programske napake, ki običajno kažejo na “hrošče” npr. indeks polja izven ...

15

Obvladovanje izjem

try - definira blok kode, za katerega želimo obvladovati izjeme

catch - definira izjemo, ki jo želimo obvladati in
definira blok kode, ki za to poskrbi

finally - definira blok kode, ki se v vsakem primeru izvrši
ne glede na to, ali se je izjema pojavila ali ne

16

Princip obravnave izjem

```
try {
    // V splošnem se ta del programa izvede normalno.
    // Lahko pa pride do izjeme ali prekinitve s stavkom break, continue ali return.

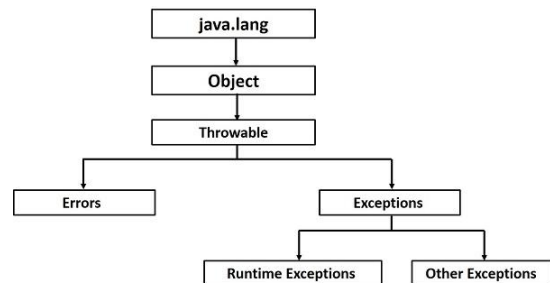
} catch (TipIzjeme1 e1) {
    // Obravnava izjeme tipa TipIzjeme1 ali podrazreda tega tipa
} catch (TipIzjeme2 e2) {
    // Obravnava izjeme tipa TipIzjeme2 ali podrazreda tega tipa
}
finally {
    // Koda se izvede vsakič, ko zapustimo del označen s try ne glede na način.
    // Ločimo več vrst zaključitev izvajanja:
    // normalno, ko dosežemo konec bloka
    // z izjemo, ki smo jo ulovili s stavkom catch
    // z izjemo, ki je nismo ujeli
    // zaradi stavka break, continue ali return
}

// vsak try ima vsaj en catch blok ali finally blok, razen try-with-resources
```

17

Proženje izjeme

- ko se pojavi izjema znotraj metode oz. bloka stavkov, se izvajanje bloka/ metode **prekine**, **kreira** ustrezeni **objekt** (exception object), ki vsebuje **informacijo o izjemi**, vključno s tipom in stanjem (npr. mesto izvajanja) programa, ko je izjema nastopila.



18

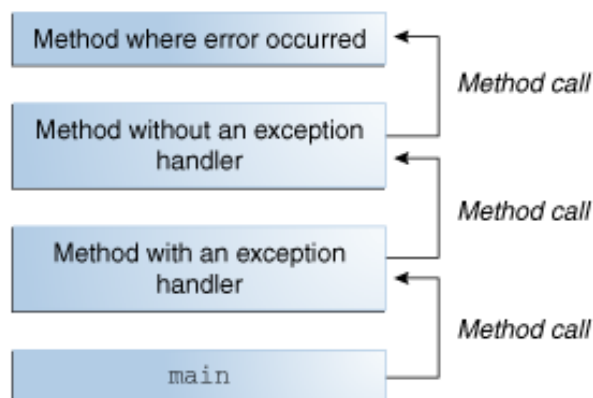
Izjeme

- ko se pojavi izjema, metoda kreira objekt izjemo in ga **preda RunTime sistemu**
- objekt izjema vsebuje informacije o izjemi, tipu in stanju programa ob pojavu izjeme
- RT sistem je odgovoren, da najde kodo, ki bo izjemo obdelala
- sledenje **skladu (verigi) klicev**
- če ne najde, se izvajanje RT in posledično Java programa zaključi

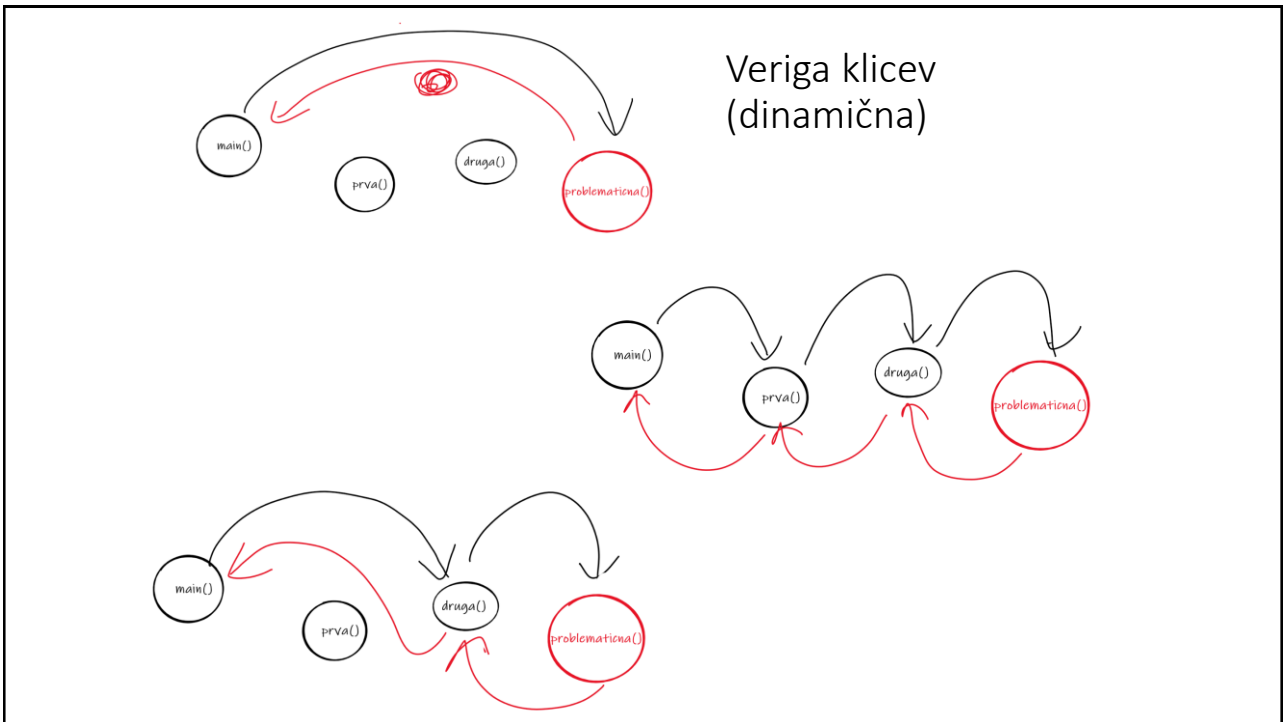
throwing
handling

19

Sklad klicev (Call Stack)



20



21

Najava proženja izjem

- Vse izjeme, razen podrazredov `RuntimeException` in `Error` imenujemo preverjane/označene/ najavljene izjeme (checked exceptions). Metoda, ki jo proži, mora to izjemo deklarirati s pomočjo `throws` v deklaraciji metode.
- Klicatelj mora vedeti, katere izjeme lahko pričakuje in mora biti sposoben reagirati - imam dve možnosti: (`try, catch`) ali pa metoda deklarira proženje te izjeme (prenese odgovornost naprej).
- `RuntimeException` (RTE) in `Error` ni potrebno loviti in niso (nujno) dokumentirane v signaturi metod

22

```

public FileReader (String imeZbirke) throws
    FileNotFoundException;

FileReader fr = null;
try {
    fr = new FileReader ("abc.txt");
} catch (FileNotFoundException fnf) {
    System.out.println("Zbirke ni! Preveri ime!");
}

```

23

Primer

```

try {
    vhodna_zbirka = args[0];
    izhodna_zbirka = args[1];
    fin = new FileInputStream(vhodna_zbirka);
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Sintaksa: java Prepisi [zbirka1] [zbirka2]");
} catch (FileNotFoundException e) {
    System.out.println("Zbirke " + vhodna_zbirka + " ne morem odpret!");
} finally {
    if (st_zlogov > 0)
        System.out.println("Prepisanih " + st_zlogov + " zlogov.");
}

```

24

Razlika med throw in throws

• throw

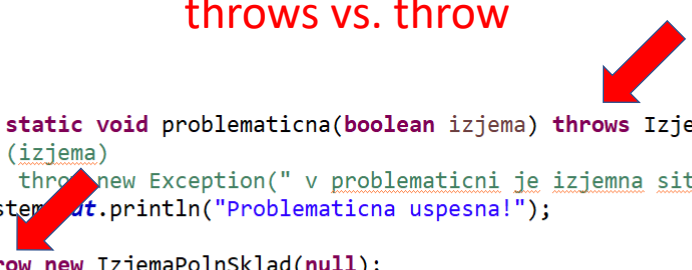
- označuje del, ki sproži izjemo
- sintaksa: `throw izraz`
- izraz je primerek razreda `Throwable` ali njegovega podrazreda

throws

- za metodo označi možne izjeme
- ne vključuje napak (`Error`)

25

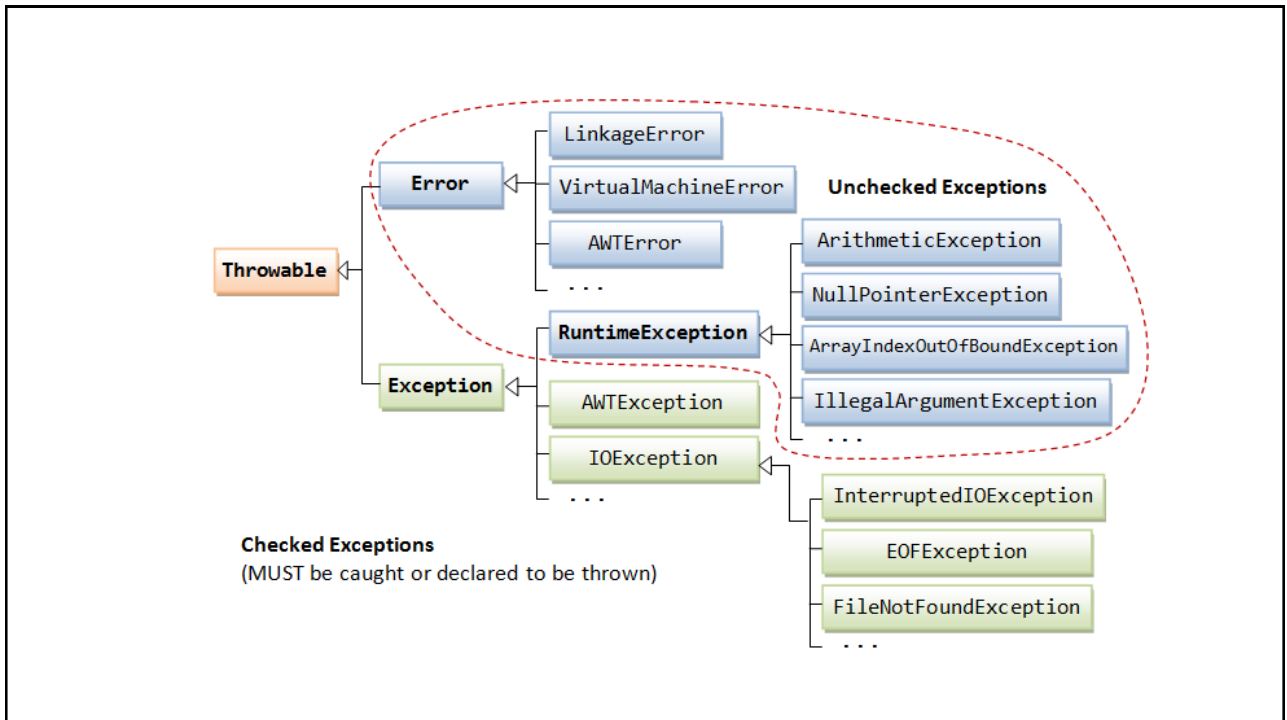
throws vs. throw



```
public static void problematicna(boolean izjema) throws IzjemaPolnSklad {
    // if (izjema)
    //     throw new Exception(" v problematicni je izjemna situacija");
    System.out.println("Problematicna uspesna!");
    throw new IzjemaPolnSklad(null);
}

public static void druga () throws IzjemaPolnSklad {
    try {
        problematicna(false);
    } catch (Exception e) {
        System.out.println("Druga popravi samo delno ");
        // e.fillInStackTrace();
        throw e; // rethrowing
    }
    System.out.println("Druga uspesna!");
}
```

26



27

```

try {

    System.out.println(10/del);
    System.out.println(cuden.toString());
    System.out.println(args[0]);

} catch (ArithmeticException e1) {
    System.out.println(" izjema aritmetična");
} catch (NullPointerException e2) {
    System.out.println(" null");
} catch (ArrayIndexOutOfBoundsException e3) {
    System.out.println(" indeks ");
}

} catch (ArithmeticException e1) {
    System.out.println(" izjema aritmetična");
} catch (NullPointerException e2) {
    System.out.println(" null");
} catch (ArrayIndexOutOfBoundsException e3) {
    System.out.println(" indeks ");
} catch (FileNotFoundException e5) {
    //..
} catch (IOException e4) {
    //..
} catch (RuntimeException rte) {
    //..
} catch (Exception ee) {
}

```

PAZIMO NA ZAPOREDJE - Najprej specifične, nato splošne!!!

28

107
108
109
110
111
112
113
114
115
116
117
118
119

PAZIMO NA ZAPOREDJE - Najprej specifične, nato splošne!!!

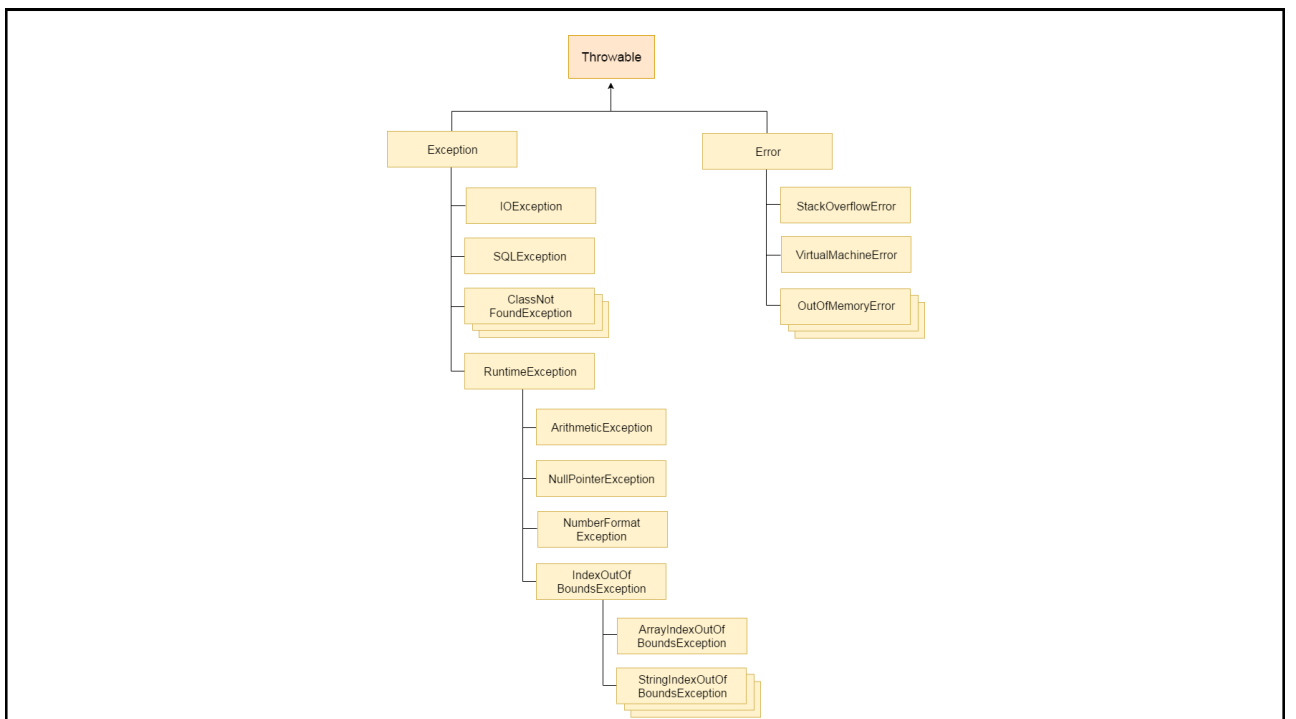
```

    } catch (ArithmeticException e1 ) {
        System.out.println(" izjema arit
    } catch (NullPointerException e2) {
        System.out.println(" null");
    } catch (ArrayIndexOutOfBoundsException e3) {
        System.out.println(" indeks ");
    } catch (IOException e4) {
        //..
    } catch (FileNotFoundException e5) {
        //..
    }

```

Unreachable catch block for FileNotFoundException. It is already handled by the catch block for IOException

29



30

Načeloma lahko obravnavam več izjem naenkrat
(ni pa pametno)

```
try {
    ...
} catch (Exception e) {...}
    raje:
try {
    ...
} catch (IOException e1) {...}
    catch (ArrayIndexOutOfBoundsException e2) {...}

    hitro namreč "obvladamo" še kaj drugega
```

31

Catch z več izjemami

```
try {
    // stuff
} catch (Exception1 | Exception2 ex) {
    // Handle both exceptions
}
```

```
catch (IOException | SQLException ex) {
    logger.log(ex);
    throw ex;
}
```

Tip **ex je najbolj specializiran skupen nadtip navedenih izjem!**

32

Pomembne metode razreda Izjema

- `catch (SomeThrowableObject ime)`
`Throwable` ima dva podrazreda `Exception` in `Error`

`java.lang.ArrayIndexOutOfBoundsException:`

`at Prepisi2.main(Prepisi2.java:14)`

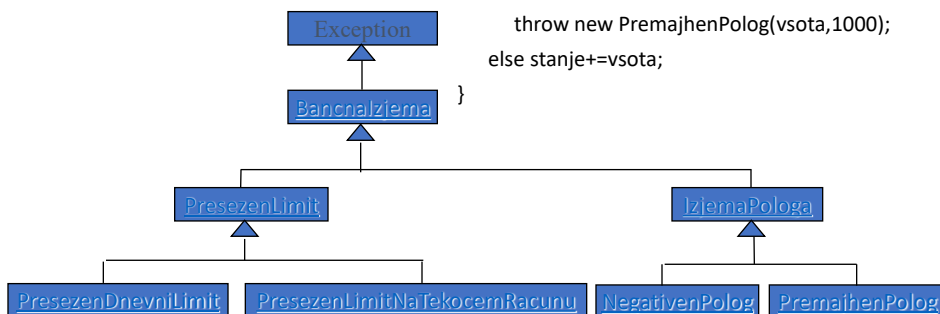
- `getMessage()` sporočilo
- `printStackTrace()` nit klicev
- `toString()` kratko sporočilo

33

Uporabniške izjeme

- definira programer
- zahtevajo dodatno delo
- poenostavijo programiranje

```
public void polog(float vsota) throws
PremajhenPolog, NegativenPolog {
    if (vsota<0)
        throw new NegativenPolog(vsota);
    else
        if ((vsota<1000)&& !(vsota<0))
            throw new PremajhenPolog(vsota,1000);
        else stanje+=vsota;
}
```



34

try-with-resources

```
try (BufferedReader br = new BufferedReader(new FileReader("a.txt"))) {

} finally {

}
```

za vire, ki implementirajo `java.io.AutoCloseable` (tudi vsi `java.io.Closeable`)

35

java.lang

Interface AutoCloseable

All Known Subinterfaces:

AsynchronousByteChannel, AsynchronousChannel, BaseStream<T,S>, ByteChannel, CachedRowSet, CallableStatement, Channel, Clip, Closeable, Connection, DataLine, DirectoryStream<T>, DoubleStream, FilteredRowSet, GatheringByteChannel, ImageInputStream, ImageOutputStream, InterruptibleChannel, IntStream, JavaFileManager, JDBCRowSet, JMXConnector, JoinRowSet, Line, LongStream, MidiDevice, MidiDeviceReceiver, MidiDeviceTransmitter, Mixer, MulticastChannel, NetworkChannel, ObjectInput, ObjectOutput, Port, PreparedStatement, ReadableByteChannel, Receiver, ResultSet, RMIConnection, RowSet, ScatteringByteChannel, SecureDirectoryStream<T>, SeekableByteChannel, Sequencer, SourceDataLine, StandardJavaFileManager, Statement, Stream<T>, SyncResolver, Synthesizer, TargetDataLine, Transmitter, WatchService, WebRowSet, WritableByteChannel

All Known Implementing Classes:

AbstractInterruptibleChannel, AbstractSelectableChannel, AbstractSelector, AsynchronousFileChannel, AsynchronousServerSocketChannel, AsynchronousSocketChannel, AudioInputStream, BufferedInputStream, BufferedOutputStream, BufferedReader, BufferedWriter, ByteArrayInputStream, ByteArrayOutputStream, CharArrayReader, CharArrayWriter, CheckedInputStream, CheckedOutputStream, CipherInputStream, CipherOutputStream, DatagramChannel, DatagramSocket, DataInputStream, DataOutputStream, DeflaterInputStream, DeflaterOutputStream, DigestInputStream, DigestOutputStream, FileCacheImageInputStream, FileCacheImageOutputStream, FileChannel, FileImageInputStream, FileImageOutputStream, FileInputStream, FileLock, FileOutputStream, FileReader, Filesystem, FileWriter, FilterInputStream, FilterOutputStream, FilterReader, FilterWriter, Formatter, ForwardingJavaFileManager, GZIPInputStream, GZIPOutputStream, ImageInputStreamImpl, ImageOutputStreamImpl,InflaterInputStream, InflaterOutputStream, InputStream, InputSteam, InputSteam, InputSteamReader, JarFile, JarInputStream, JarOutputStream, LineNumberInputStream, LineNumberReader, LogStream, MemoryCacheImageInputStream, MemoryCacheImageOutputStream, Mlet, MulticastSocket, ObjectInputStream, ObjectOutputStream, OutputStream, OutputStream, OutputStream, OutputStreamWriter, Pipe.SinkChannel, Pipe.SourceChannel, PipedInputStream, PipedOutputStream, PipedReader, PipedWriter, PrintStream, PrintWriter, PrivateMlet, ProgressMonitorInputStream, PushbackInputStream, PushbackReader, RandomAccessFile, Reader, RMIConnectionImpl, RMIConnectionImpl.Stub, RMIConnector, RMIIOPServerImpl, RMJRMPServerImpl, RMIServerImpl, Scanner, SelectableChannel, Selector, SequenceInputStream, ServersSocket, ServerSocketChannel, Socket, SocketChannel, SSLServerSocket, SSLSocket, StringBufferInputStream, StringReader, StringWriter, URLClassLoader, Writer, XMLDecoder, XMLEncoder, ZipFile, ZipInputStream, ZipOutputStream

public interface **AutoCloseable**

An object that may hold resources (such as file or socket handles) until it is closed. The `close()` method of an `AutoCloseable` object is called automatically when exiting a try-with-resources block for which the object has been declared in the resource specification header. This construction ensures prompt release, avoiding resource exhaustion exceptions and errors that may otherwise occur.

API Note:

It is possible, and in fact common, for a base class to implement `AutoCloseable` even though not all of its subclasses or instances will hold releasable resources. For code that must operate in complete generality, or when it is known that the `AutoCloseable` instance requires resource release, it is recommended to use try-with-resources constructions. However, when using facilities such as `Stream` that support both I/O-based and non-I/O-based forms, try-with-resources blocks are in general unnecessary when using non-I/O-based forms.

Since:

1.7

Method Summary

All Methods		Instance Methods		Abstract Methods	
Modifier and Type		Method and Description			
void		<code>close()</code>		Closes this resource, relinquishing any underlying resources.	

36

PRENOS ARGUMENTOV

37

Podajanje argumentov

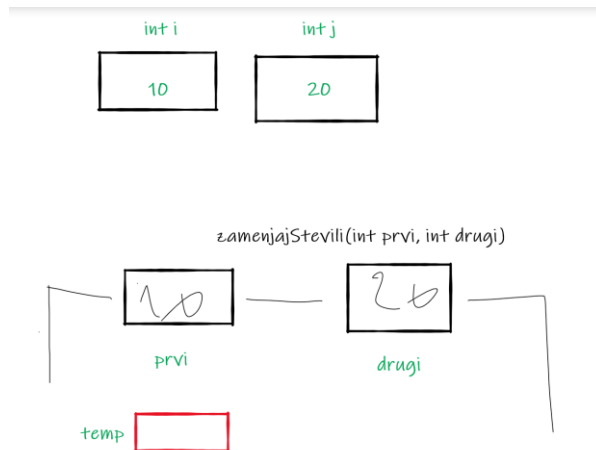
- v primeru osnovnih podatkovnih tipov se argumenti podajajo po vrednosti
- posledično se vsi argumenti kopirajo oz. imamo kopijo vrednosti
- objekti se podajajo po referenci

PAZI:

prenese se **kopija reference po vrednosti** (by value)

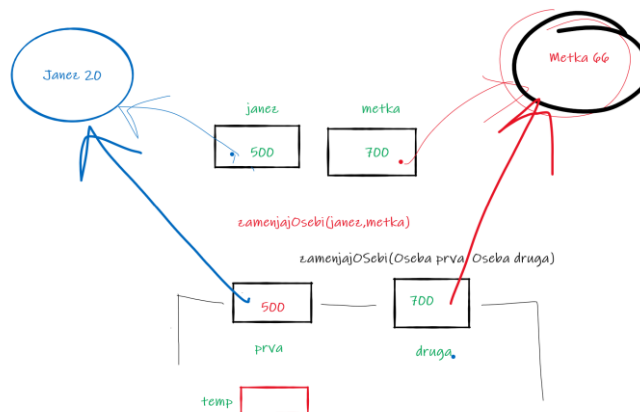
38

Osnovni tipi po vrednosti



39

Objekti – reference po vrednosti



40